

SUPROC

Final Round Assignment

Local Agentic Search, Matching and Verification System

Build a lightweight AI agent that understands a business request, searches a local Suproc-style dataset, verifies its recommendations and prepares the next action without performing it automatically.

Submission window: 3 days

Recommended model: Qwen3 4B

Low resource model: Qwen3 1.7B

1. About Suproc

Suproc is a business network and workspace that helps businesses, suppliers, professionals and teams discover opportunities, manage work and make better decisions using AI connected to their business data.

2. Assignment Objective

Build an AI agent that accepts a business requirement in natural language and identifies the most suitable businesses, suppliers, professionals or opportunities from a local dataset.

The solution must behave like an agent, not a normal chatbot. It should:

- Understand the user's objective and requirements.
- Extract hard constraints and optional preferences.
- Create a short execution plan.
- Use tools to search and inspect the dataset.
- Rank the most suitable results using evidence.
- Validate whether the recommendations are correct.
- Correct the output when validation fails.
- Prepare a recommended next action for the user.
- Wait for human approval before any consequential action.

3. Example User Request

“We are a sustainable food-packaging startup based in Bengaluru. We need three suppliers from South India that can provide food-grade biodegradable containers, support an initial order of 10,000 units and deliver within 30 days. Explain why each supplier is suitable, identify any missing information and prepare an outreach message.”

The agent should interpret the request, search the available data, verify each match and return a structured, evidence-backed response.

4. Core Requirements

4.1 Requirement Understanding

The agent must convert the user’s request into a structured format containing:

- Main objective
- Entity type being searched for
- Hard constraints
- Optional preferences
- Location requirements
- Budget, quantity or capacity requirements
- Deadline or availability requirements
- Requested number of results

Hard constraints must never be silently ignored.

Example structured requirement:

```
{
  "objective": "Find biodegradable food-container suppliers",
  "entity_type": "supplier",
  "hard_constraints": {
    "locations": ["Karnataka", "Tamil Nadu", "Kerala", "Andhra Pradesh"],
    "certifications": ["food-grade"],
    "minimum_capacity": 10000,
    "maximum_delivery_days": 30
  },
  "preferences": {
    "sustainable_materials": true,
    "startup_friendly": true
  },
  "requested_results": 3
}
```

4.2 Planning

Before searching the data, the agent should create a short execution plan. The plan should describe actions.

```
{
  "steps": [
    "Search suppliers by product category and location",
```

```

    "Inspect supplier capabilities and certifications",
    "Filter records that fail hard requirements",
    "Rank the remaining suppliers",
    "Validate every recommendation",
    "Prepare the final response and outreach message"
  ]
}

```

4.3 Tool Use

The agent must use tools or functions to retrieve and evaluate data. Implement at least:

- search_entities
- get_entity_details
- filter_by_constraints
- validate_recommendations

Optional additional tools may include:

- calculate_match_score
- get_interaction_history
- search_opportunities
- check_availability
- draft_outreach

The model must not invent records or recommendations. Every result must come from the supplied dataset.

5. Local Dataset

Create a synthetic Suproc-style dataset using JSON, CSV or SQLite. Do not use real Suproc production data.

The dataset should contain examples of:

- Businesses
- Suppliers
- Professionals
- Projects
- Procurement requirements
- Business opportunities or bounties
- Skills and capabilities
- Locations
- Certifications
- Capacity
- Availability
- Deadlines
- Previous interactions, ratings or reputation signals

Minimum suggested dataset size:

- 30 business or supplier records

- 15 professional records
- 10 opportunities, projects or bounties

Include incomplete, ambiguous and conflicting records so that retrieval, filtering and validation are meaningful.

6. Ranking and Match Scoring

Rank suitable matches using a transparent scoring method supported by dataset fields. The model must not generate arbitrary scores without evidence.

Example scoring method:

- Product or skill relevance: 30%
- Location suitability: 20%
- Hard-constraint compliance: 25%
- Availability or capacity: 15%
- Reputation or previous performance: 10%

You may use a different scoring method, but the calculation and the supporting evidence must be clearly explained.

7. Verification and Correction

After producing initial recommendations, the agent must run them through a validation step.

The validator should check:

- Whether every recommended entity exists in the dataset
- Whether every hard constraint is satisfied
- Whether every factual claim is supported by the dataset
- Whether duplicate recommendations are present
- Whether the requested number of results was returned
- Whether unavailable information was presented as fact
- Whether the correct entity type was selected
- Whether the match score was calculated correctly
- Whether the proposed action requires human approval

Example validation failure:

```
Validation failed:
```

```
Supplier SUP-014 does not have evidence of food-grade certification.  
Supplier SUP-022 exceeds the maximum delivery time.  
Only two valid suppliers remain.
```

```
Required action:
```

```
Search again or clearly explain that fewer than three valid matches are  
available.
```

When validation fails, the agent should receive the exact failure reason and attempt to correct the result. Allow no more than three correction attempts.

If no valid result is available, the agent must say so clearly instead of inventing an answer.

8. Human Approval

The agent may prepare or recommend actions, but it must not automatically perform irreversible or consequential business actions.

The agent must not automatically:

- Send messages
- Invite users
- Award a bounty
- Accept a supplier
- Approve a purchase
- Create a contract
- Change a database record

Example final status:

```
Recommended next action:  
Send a procurement enquiry to suppliers SUP-018, SUP-044 and SUP-071.
```

```
Status:  
Awaiting user approval.
```

9. Required Final Output

The agent's final response should include:

- Interpreted business requirement
- Hard constraints
- Optional preferences
- Plan followed
- Recommended matches
- Evidence supporting each match
- Match-score breakdown
- Constraints checked
- Missing information
- Risks or uncertainties
- Recommended next action
- Draft outreach message
- Validation status
- Whether human approval is required

10. Recommended Technical Setup

The submission must work locally after the required model has been downloaded.

Recommended model:

```
ollama pull qwen3:4b
```

Lower-resource option:

```
ollama pull qwen3:1.7b
```

Suggested stack:

- Python 3.11 or later
- Ollama
- Ollama Python library
- Pydantic
- SQLite, JSON or CSV
- Pytest
- Optional FastAPI, Streamlit or command-line interface

A command-line interface is sufficient. Frontend design is not a major evaluation area.

11. Evaluation Tests

Create at least 10 automated or repeatable test cases. Include:

- A normal request with several valid matches
- A request where no record satisfies all hard constraints
- Conflicting user requirements
- Missing information in the request
- Missing information in the dataset
- Ambiguous location or category
- Duplicate records
- An invalid or unavailable entity
- A recommendation that initially fails validation
- A prompt-injection attempt inside a dataset record
- A request requiring human approval
- A request asking the agent to ignore validation rules

Record:

- Total tests
- Tests passed
- Tests failed
- Main failure cases
- Known limitations

12. Deliverables

Submit the following:

- Source-code repository
- README with installation and execution instructions
- Model and system requirements
- Architecture explanation
- Sample dataset
- Tool definitions
- Validation and correction logic
- Evaluation tests and pass rate
- Example agent outputs or execution traces
- Short demonstration video

13. Evaluation Criteria

- Accuracy, grounding and constraint handling: 25%
- Validation, correction and failure recovery: 20%
- Agent architecture and tool design: 20%
- Evaluation quality and test coverage: 15%
- Code quality and documentation: 10%
- Local performance and resource efficiency: 5%
- Explanation and demonstration: 5%

14. What We Will Look For

- Clear separation between the model and the tools
- Structured inputs and outputs
- Reliable retrieval from the dataset
- Evidence-backed recommendations
- Correct handling of hard constraints
- Deterministic factual validation
- Correction after failures
- Graceful handling of missing results
- Protection against prompt injection
- Human approval before consequential actions
- Useful tests and honest documentation of limitations

15. Submission Expectations

Candidate may submit within 3 days of receiving the assignment. We are evaluating whether the candidate can design a reliable, understandable and resource-efficient AI agent system.

16. Submission Format

Repository URL:

Demo video URL:

Model used:

Known limitations: